INFORME TÉCNICO DE REVISIÓN FINAL · CRM SALVADORA

Administración Completa de WhatsApp: Proveedores, QR Evolution API v2.3.7, Envío Manual Admin, Adjuntos y CLI Python

Consolidación integral sobre Fase 1 + Fase 2 · Fecha: 20 de octubre de 2026 · Servidor DGX Spark (Evolution API + CRM)

SUITES DE TEST BACKEND	TESTS WHATSAPP / EVOLUTION	BUILD BACKEND & FRONTEND	BASE DE DATOS / DATOS
38 / 38 PASS (306 tests)	**29 escenarios PASS**	**EXIT CODE 0 (Limpio)**	**100% Intactos (0 resets)**

1. Resumen de lo reutilizado (Fase 1 y Fase 2)

Se inspeccionó el repositorio completo y se conservó toda la arquitectura ya operativa sin duplicar servicios ni pantallas:

- **Entidades y Migración Aditiva**: `AppSettings.whatsappAutomationPaused` y `AgentConfig` (`whatsappProvider`, credenciales de Evolution API y campos preparados de Meta Cloud API) junto con la migración `1782912000000-Phase10WhatsappGatewayAndPause.ts`.
- **Gateway y Proveedores**: `WhatsappGatewayService`, `YCloudProvider` y `EvolutionProvider`.
- **Webhooks y Control de Instancia**: `EvolutionWebhookController` (validación fail-closed de `x-evolution-secret` e instancia, deduplicación por `externalId`, estados de entrega) y `EvolutionAdminController`.
- **Pausa Global y Human Handoff**: coexistencia entre `whatsappAutomationPaused` (global) y `conversation.handoff` (por hilo).

2. Archivos Creados

Archivo	Responsabilidad
`backend/src/whatsapp/whatsapp-attachment-validator.ts`	Validador estricto de archivos adjuntos (`.pdf`, `.docx`, `.xlsx`, `.txt`, `.mp3`, `.mp4`) por extensión, tipo MIME, *magic bytes*, inspección interna del contenedor ZIP OpenXML (`extractZipEntryNames`) y tamaño máximo configurable `WHATSAPP_MAX_ATTACHMENT_MB` (`413 Payload Too Large`).
`backend/src/whatsapp/whatsapp-admin-send.service.ts`	Servicio central de envío manual (UI Admin y CLI Python) con soporte de solo texto, solo archivo, texto + archivo y doble envío secuencial para texto + `.mp3`, persistencia en Contacto/Conversación/Mensajes, emisión SSE y auditoría `whatsapp.admin_send`.
`backend/src/whatsapp/whatsapp-send.controller.ts`	Controlador con `POST /api/whatsapp/admin/send` (protegido con `JwtAuthGuard` + `RolesGuard` + `@Roles(UserRole.ADMIN)`) y `POST /api/whatsapp/cli/send` (protegido con `x-whatsapp-cli-token` mediante `crypto.timingSafeEqual` fail-closed).
`tools/whatsapp_send.py`	Herramienta CLI en Python para el DGX que envía mensajes y archivos manuales invocando `POST /api/whatsapp/cli/send` mediante `multipart/form-data` sin conectarse directamente a PostgreSQL ni saltarse el CRM.

3. Archivos Modificados

Archivo	Cambios Aplicados
`backend/src/audit/audit.types.ts`	Añadida la constante de auditoría `WHATSAPP_ADMIN_SEND: 'whatsapp.admin_send'`.
`backend/src/agents/agents-config.service.ts`	Guardrail en `update()` que bloquea activar `meta` como proveedor operativo mientras `MetaProvider` no esté implementado, y preserva las credenciales de los demás proveedores al alternar entre `evolution` y `ycloud`.
`backend/src/whatsapp/providers/evolution.provider.ts`	Implementado `sendMediaMessage()` (`POST /message/sendMedia/{instance}`) convirtiendo el `Buffer` a Base64 puro únicamente en el momento de enviar a Evolution API, y limpieza de `qrBase64` cuando la instancia está en estado `open`.
`backend/src/whatsapp/whatsapp-gateway.service.ts`	Añadidos los métodos `sendManualMediaMessage()` y `sendAutomatedMediaMessage()` enrutados al proveedor activo.
`backend/src/whatsapp/whatsapp.module.ts`	Registrados y exportados `WhatsappSendController` y `WhatsappAdminSendService`.
`frontend/src/app/settings/page.tsx`	Selector `[ Evolution API ]`, `[ YCloud ]`, `[ Meta API Oficial ]`, tarjeta propia personalizada para el QR real de Evolution API, estados visuales claros y tarjeta Admin **"Enviar WhatsApp (Envío Manual Admin)"**.
`frontend/src/app/agents/[agentKey]/page.tsx`	Alineado el orden de proveedores y la protección de guardado cuando se visualiza la pestaña preparada de Meta API Oficial.
`backend/src/whatsapp/evolution-phase2.spec.ts`	Ampliada la suite de tests unitarios hasta cubrir los 29 casos de prueba exigidos.

4. Selector de Proveedor (`Evolution / YCloud / Meta`)

- Opciones visuales en orden: `[ Evolution API ]`, `[ YCloud ]`, `[ Meta API Oficial ]`.
- Muestra únicamente los campos del proveedor seleccionado y el botón **"Guardar proveedor y configuración"**.
- Cambiar entre `evolution` y `ycloud` conserva intactas las credenciales guardadas de los demás proveedores.
- Al visualizar **Meta API Oficial**, se muestra claramente "*Meta API Oficial — preparado / pendiente de activación*": permite dejar pre-guardadas las credenciales de Meta sin activar `meta` como proveedor operativo hasta que `MetaProvider` esté habilitado.

5. Tarjeta Personalizada del QR Real de Evolution API

Diseño integrado en `/settings` (cuando `qrBase64` está disponible y `state !== 'open'`):

- **Cabecera**: `Conectar WhatsApp`
- **Metadatos**: `Agente: <nombre>` · `Número: +34 646 034 231` · `Instancia: salvadora_whatsapp`
- **Estado**: 🟠 Esperando vinculación (badge general: ESPERANDO QR)
- **Imagen QR**: renderizada directamente desde `qrBase64` devuelto por Evolution API (`GET /instance/connect/{instance}`), centrada, amplia (`h-64 w-64`), sobre fondo blanco, sin recortes ni logos superpuestos que alteren los módulos del QR.
- **Instrucciones paso a paso**: *Abre WhatsApp en el teléfono → Ajustes / Dispositivos vinculados → Vincular dispositivo → Escanea este código.*
- **Acciones**: botones `[ Actualizar QR ]` y `[ Actualizar estado ]`.
- **Al conectar (`state === 'open'`)**: oculta el QR automáticamente, muestra 🟢 CONECTADO, el número conectado y los botones `Actualizar estado` y `Desconectar`.

6. Utilidad Admin "Enviar WhatsApp" (Texto y Archivos)

- Tarjeta exclusiva para rol `admin` en **Ajustes (`/settings`)** → *Enviar WhatsApp (Envío Manual Admin)*.
- Campos: **Agente** (por defecto `booking`), **Teléfono destino** (`+34 ...`), **Mensaje / texto** (textarea opcional si hay archivo) y **Archivo adjunto** (máx. 1 archivo).
- Envía `multipart/form-data` al endpoint protegido `POST /api/whatsapp/admin/send`.

7. Tipos de Archivo Soportados y Validación Estricta

Extensión	MIME Esperado	Categoría	Validación de Cabecera / Estructura Interna
`.pdf`	`application/pdf`	`document`	Verifica *magic bytes* `%PDF-` al inicio del archivo.
`.docx`	`application/vnd.openxmlformats-officedocument.wordprocessingml.document`	`document`	Verifica contenedor ZIP (`PK\x03\x04` + `PK\x05\x06`) y exige entradas internas `[Content_Types].xml`, carpeta `word/` y `word/document.xml`. Rechaza ZIP comunes o `.xlsx` renombrados.
`.xlsx`	`application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`	`document`	Verifica contenedor ZIP (`PK\x03\x04` + `PK\x05\x06`) y exige entradas internas `[Content_Types].xml`, carpeta `xl/` y `xl/workbook.xml`. Rechaza ZIP comunes o `.docx` renombrados.

Extensión	MIME Esperado	Categoría	Validación de Cabecera / Estructura Interna
`.txt`	`text/plain`	`document`	Verifica texto UTF-8 válido (`TextDecoder('utf-8', { fatal: true })`), sin bytes nulos `0x00` ni firmas binarias PDF/ZIP renombradas.
`.mp3`	`audio/mpeg`	`audio`	Verifica cabecera `ID3` (`0x49 0x44 0x33`) o *frame sync* MPEG audio válido.
`.mp4`	`video/mp4`	`video`	Verifica átomo ISO Base Media `ftyp` en los bytes 4..8.

8. Resolución del Caso `.mp3` + Texto

Dado que WhatsApp no muestra `caption` en archivos de audio, cuando se envía `message` + archivo `.mp3`:

1. Se envía primero el mensaje de texto mediante `sendManualTextMessage` (`POST /message/sendText/{instance}`).
2. Se envía a continuación el archivo `.mp3` como audio mediante `sendManualMediaMessage` (`POST /message/sendMedia/{instance}` con `mediatype: "audio"`).
3. Se persisten ambos mensajes salientes en la conversación de Inbox y se devuelven ambos identificadores en `providerMessageIds: [idTexto, idAudio]`.

9. Control de Tamaño Máximo (`WHATSAPP_MAX_ATTACHMENT_MB`)

El validador consulta `process.env.WHATSAPP_MAX_ATTACHMENT_MB` (valor por defecto: `16` MB). Si el archivo supera dicho umbral, devuelve **HTTP 413 Payload Too Large** con un mensaje explícito indicando el peso del archivo y el límite interno configurado en el CRM.

10 y 11. CLI Python `tools/whatsapp_send.py` y Autenticación `WHATSAPP_CLI_TOKEN`

- **Uso desde el DGX**:

```
python3 tools/whatsapp_send.py --agent booking --to +34600111222 --message "Te adjunto el horario" --
```

- **Seguridad fail-closed**: `POST /api/whatsapp/cli/send` exige la cabecera `x-whatsapp-cli-token` y la compara contra `process.env.WHATSAPP_CLI_TOKEN`. Si la variable de entorno no está definida o está vacía, rechaza siempre con `401 Unauthorized`. Comprueba primero que ambos `Buffer` tengan la misma longitud antes de invocar `crypto.timingSafeEqual`.

12, 13 y 14. Registro en Inbox, Auditoría, Pausa Global / Handoff y Preparación Meta

- **Persistencia e Inbox**: normaliza el teléfono a E.164 (`normalizePhoneStrict`), crea o reutiliza el contacto (`contactsService.upsertByPhone`) y la conversación `${agentKey}:${to}` , guarda el mensaje con `direction = 'outbound'` , `channel = 'whatsapp'` , `status = 'sent' | 'failed'` , `providerMessageId` y metadatos del adjunto, y emite eventos SSE en tiempo real.
- **Auditoría**: emite `whatsapp.admin_send` con actor, `agentKey` , `provider` , destinatario `to` , nombre/MIME/tamaño del archivo y `providerMessageIds` , sin almacenar secretos ni contenido binario/Base64.

- **Convivencia con Pausa Global y Human Handoff**: los envíos manuales (Inbox, Admin Sender y CLI Python) funcionan incluso cuando `whatsappAutomationPaused = true` o `conversation.handoff = true`.
- **Preparado para Meta Oficial**: toda la lógica de negocio e interfaces (UI y CLI) delegan en `WhatsappGatewayService`, por lo que activar `MetaProvider` en el futuro no requerirá reescribir ningún módulo del CRM.

15, 16 y 17. Validación de Tests, Builds y Estado Git

> **Resultados de validación ejecutados:**
>
> - `cd backend && pnpm test` → **38 test suites pasadas de 38** (**306 tests pasados de 306**, incluyendo los 29 tests de `src/whatsapp/evolution-phase2.spec.ts`).
> - `cd backend && pnpm build` → `nest build` completado sin errores (código 0).
> - `cd frontend && pnpm exec tsc --noEmit && pnpm build` → TypeScript y Next.js 16.2.9 build de producción completados sin errores (código 0).
> - `python tools/whatsapp_send.py --help` → verificado correctamente (código 0).